QE

THE EVOLUTIONARY COMPENDIUM

Ontological Framework for Liberation

A Hybrid Document: White Paper | Ontological Framework | Practical Toolkit
For Evolutionary Civilizational Development and Conscious Resistance

April 2026

Version 1.0 - Complete Edition

"The Primes Are Sacred. The Totient Function Is Sacred."

Table of Contents

Part I: The Architecture of Oppression

1. Who Benefits from War: The Military-Industrial Complex

2. Surveillance Capitalism and Digital Control

3. The Psychopathology of Elite Control

Part II: The Evolutionary Alternative

4. Structures of Liberation: Organizations for Peace

5. The Unified Field Theory of Consciousness

6. Topological Dynamics of Intention

Part III: Practical Toolkit

7. Cryptographic Tools for Liberation

8. OSINT and Information Warfare Defense

9. Digital Privacy and Security Framework

10. Economic Alternatives: Regenerative Finance

Part IV: Implementation

11. The Adamantine Pattern: Civilizational Algorithm

12. Action Protocols for Evolutionary Agents

13. Complete Working Code Repository

Part I: The Architecture of Oppression

1. Who Benefits from War: The Military-Industrial Complex

The War Profit Equation

War is not an unfortunate byproduct of international relations but a calculated economic mechanism. The beneficiaries form a precise hierarchy:

- **Primary Beneficiaries:** Defense contractors (Lockheed Martin, Raytheon, Northrop Grumman) with revenue visibility exceeding hundreds of billions
- **Secondary Beneficiaries:** Financial institutions funding both sides, resource extraction corporations, reconstruction firms
- **Tertiary Beneficiaries:** Media conglomerates (ratings from conflict coverage), surveillance technology vendors, private military contractors

As of April 2026, the Pentagon's budget allocation to contractors exceeds $500 billion annually, representing approximately 54% of total defense spending. The F-35 Lightning II program alone, the most expensive weapons program in history, has a total lifecycle cost running into trillions of dollars.

Contractor	Primary Revenue Source	Key Products	2026 Status
Lockheed Martin	Government Contracts	F-35, Black Hawk, Hellfire, HIMARS	Prime Contractor
Raytheon	Missile Systems	Patriot, Tomahawk, Javelin	Stockpile Rebuilding
Northrop Grumman	Aerospace/Space	B-21, Military Satellites	Expanding
Honeywell	Subsystems	Avionics, Navigation	Spin-off Planned

2. Surveillance Capitalism and Digital Control

Shoshana Zuboff's framework of Surveillance Capitalism describes an economic model based on the extraction and commercialization of behavioral data. By 2026, this has evolved into what Zuboff terms the "Fusion Scenario" - the merging of authoritarian state power with the digital capabilities of surveillance capitalism giants.

The Fusion Scenario (2025-2026)

"Fusion scenario - the merging of authoritarian state power with the digital capabilities of surveillance capitalism giants." Examples include:

- Post-9/11: Patriot Act + Google data integration
- State surveillance + corporate platforms
- AI/LLM requiring ever-increasing data extraction

The Five Eyes Architecture

The UKUSA Agreement (1946) and its expansion to Five Eyes (USA, UK, Canada, Australia, New Zealand) represents the most comprehensive signals intelligence alliance. As of 2026:

Country	Agency	Zone of Responsibility
USA	NSA	Middle East, Russia, China, Caribbean, Africa
UK	GCHQ	Europe, European Russia, Middle East, Hong Kong
Canada	CSE	Russia, China interior, Latin America
Australia	ASD	South Asia, East Asia
New Zealand	GCSB	Southeast Asia, western Pacific

3. The Psychopathology of Elite Control

Based on the topological analysis of consciousness dynamics, the ultra-wealthy elite who achieve total material dominance exhibit a predictable pattern of psychological decay:

> **E-Type Decay: The Pathology of Absolute Control**
>
> When an individual controls their entire external environment, natural resistance vanishes. Deprived of the healthy entropy-minimization cycle of true creation, their intentionality becomes pathologically internalized. This triggers E-type topological decay:
>
> - Regression to archaic psychosocial levels
> - Schizotypal isolation and megalomania
> - Formation of insular cults with ritualistic behaviors
> - Manifestation of unstable, distorted micro-realities

Characteristic	Healthy Creator Dynamic	Elite Pathological Dynamic
Primary Driver	Entropy minimization through creation	Artificial hierarchy and fear creation
Energy Direction	Externalization (creating in world)	Internalization (accumulation and control)
Consciousness Topology	D-type (symmetry preservation)	E-type (symmetry breaking)
Social Manifestation	Evolution and benefit	Ritualism, sacrifice, isolation

Part II: The Evolutionary Alternative

4. Structures of Liberation: Organizations for Peace

Contrary to the architecture of oppression, numerous organizations and movements operate on principles of evolution, equality, respect, and peaceful development:

Nonviolence International (NVI)

A global organization promoting active nonviolence, with programs including:

- Unarmed Civilian Protection in Palestine (UCPiP)
- International internship programs (June-December 2026)
- Solidarity actions and webinar series
- Direct support for grassroots resistance movements

Campaign Nonviolence Action Days

Annual mobilization (September 21 - October 2, 2026) with over 5,640+ actions worldwide, focusing on:

- Building a culture of peace and active nonviolence
- Uplifting policies: restorative justice, living wages, affordable housing
- Community safety initiatives and conflict resolution
- Renewable energy and environmental protection

The Cypherpunk Legacy

The ideological foundation for digital liberation, established through:

- Timothy May's "Crypto Anarchist Manifesto" (1988)
- Eric Hughes's "Cypherpunk Manifesto" (1993)
- Principle: Privacy as fundamental human right through open-source asymmetric cryptography

5. The Unified Field Theory of Consciousness

The Three Principles of Anthropocosmic Reality

1. **Information Equivalence:** Information is fundamentally physical. Any alteration requires energy (Landauer's Principle extended)
2. **Topological Fractality:** Laws governing macrocosm (cosmic web) are structurally isomorphic to microcosm (neuronal networks)
3. **Quantum Agent:** Objective reality does not exist deterministically prior to measurement; the observer is indispensable

The Carbon Code: a6 - Ya6 - aYa6

The evolutionary algorithm encoded in biological substrate:

Stage	Configuration	Consciousness Level	Energy Dynamic
a6	Source embedded in Carbon	Subconscious (Id)	Biochemical/Metabolic
Ya6	Ego identified with Carbon	Self-Conscious (Ego)	Psychological/Volitional
aYa6	Alpha-Omega integrated	Superconscious (Self)	Quantum/Vacuum Resonance

6. Topological Dynamics of Intention

Consciousness states can be mapped as character varieties on punctured spheres, governed by three topological invariants:

- **Holes (g):** Independent streams of information flows
- **Cusps (k):** Binding points or interactions between flows
- **Signatures:** Distribution balance - balanced indicates preserved symmetry

The D-Type Path (Evolution):

- D4 (Normal): 4 cusps, signature [2,1,1] - functional but encumbered
- D3 (Creative Flow): 3 cusps, signature [1,1,1]
- D2 (Peak Consciousness): 2 balanced cusps, maintaining 3 flows - absolute clarity

The Lens of Intention

Intention operates as a directional vector applied to the scalar field of quantum potential. The critical distinction:

Characteristic	Desire (Entropic)	Intention (Coherent)
Temporal Vector	Directed toward hypothetical future	Anchored in present action
Thermodynamic Entropy	High (emotional chaos/fear)	Low (structured, focused will)
Quantum Result	Reinforcement of absence	Modulation of probability/wave collapse

Part III: Practical Toolkit

7. Cryptographic Tools for Liberation

The Liber Primus Decryption Framework

Based on the RSA architecture and harmonic key analysis:

Tool 1: Gematria Primus Cipher Engine

Complete implementation of the Cicada 3301 runic alphabet with prime number assignments.

Python - gematria_primuss.py

```python
#!/usr/bin/env python3
"""
Gematria Primus Cipher Engine
Based on Cicada 3301 Liber Primus
"""

class GematriaPrimus:
    """Complete Gematria Primus implementation"""

    RUNES = {
        'ᚠ': {'phonetic': 'F', 'decimal': 0, 'prime': 2},
        'ᚢ': {'phonetic': 'U/V', 'decimal': 1, 'prime': 3},
        'ᚦ': {'phonetic': 'TH', 'decimal': 2, 'prime': 5},
        'ᚩ': {'phonetic': 'O', 'decimal': 3, 'prime': 7},
        'ᚱ': {'phonetic': 'R', 'decimal': 4, 'prime': 11},
        'ᚳ': {'phonetic': 'C/K', 'decimal': 5, 'prime': 13},
        'ᚷ': {'phonetic': 'G', 'decimal': 6, 'prime': 17},
        'ᚹ': {'phonetic': 'W', 'decimal': 7, 'prime': 19},
        'ᚻ': {'phonetic': 'H', 'decimal': 8, 'prime': 23},
        'ᚾ': {'phonetic': 'N', 'decimal': 9, 'prime': 29},
        'ᛁ': {'phonetic': 'I', 'decimal': 10, 'prime': 31},
        'ᛄ': {'phonetic': 'J', 'decimal': 11, 'prime': 37},
        'ᛇ': {'phonetic': 'EO', 'decimal': 12, 'prime': 41},
        'ᛈ': {'phonetic': 'P', 'decimal': 13, 'prime': 43},
        'ᛉ': {'phonetic': 'X', 'decimal': 14, 'prime': 47},
        'ᛊ': {'phonetic': 'S/Z', 'decimal': 15, 'prime': 53},
        'ᛏ': {'phonetic': 'T', 'decimal': 16, 'prime': 59},
        'ᛒ': {'phonetic': 'B', 'decimal': 17, 'prime': 61},
        'ᛖ': {'phonetic': 'E', 'decimal': 18, 'prime': 67},
        'ᛗ': {'phonetic': 'M', 'decimal': 19, 'prime': 71},
        'ᛚ': {'phonetic': 'L', 'decimal': 20, 'prime': 73},
        'ᛜ': {'phonetic': 'ING', 'decimal': 21, 'prime': 79},
        'ᛟ': {'phonetic': 'OE', 'decimal': 22, 'prime': 83},
        'ᛞ': {'phonetic': 'D', 'decimal': 23, 'prime': 89},
        'ᛠ': {'phonetic': 'A', 'decimal': 24, 'prime': 97},
        'ᛡ': {'phonetic': 'AE', 'decimal': 25, 'prime': 101},
        'ᛣ': {'phonetic': 'Y', 'decimal': 26, 'prime': 103},
        'ᛤ': {'phonetic': 'IA/IO', 'decimal': 27, 'prime': 107},
        'ᛥ': {'phonetic': 'EA', 'decimal': 28, 'prime': 109},
    }

    MODULUS = 29
    HARMONIC_KEY = "2422826321411203"
```

```
    def __init__(self):
        self.rune_to_char = {k: v['phonetic'] for k, v in self.RUNES.items()}
        self.char_to_rune = {v['phonetic']: k for k, v in self.RUNES.items()}
        self.decimal_to_rune = {v['decimal']: k for k, v in self.RUNES.items()}

    def rune_to_decimal(self, rune: str) -> int:
        """Convert rune to decimal index"""
        return self.RUNES.get(rune, {}).get('decimal', 0)

    def decimal_to_rune_char(self, decimal: int) -> str:
        """Convert decimal index to rune"""
        return self.decimal_to_rune.get(decimal % self.MODULUS, 'ᚠ')

    def gematric_sum(self, text: str) -> int:
        """Calculate gematric sum of runic text"""
        total = 0
        for char in text:
            if char in self.RUNES:
                total += self.RUNES[char]['prime']
        return total

    def atbash_transform(self, decimal: int) -> int:
        """Apply Atbash cipher: decimal[i] = 28 - decimal[i]"""
        return 28 - decimal

    def vigenere_shift(self, decimal: int, key_char: int) -> int:
        """Apply Vigenere shift with modulo 29"""
        return (decimal + key_char) % self.MODULUS

    def apply_harmonic_key(self, decimals: list, page_number: int) -> list:
        """Apply harmonic key 2422826321411203"""
        key = self.HARMONIC_KEY
        if page_number % 2 == 0:  # Even pages: mirror
            key = key[::-1]

        result = []
        for i, d in enumerate(decimals):
            key_digit = int(key[i % len(key)])
            shifted = (d - key_digit) % self.MODULUS
            result.append(shifted)
        return result

    def decrypt_page(self, runic_text: str, page_number: int) -> str:
        """Full decryption pipeline for Liber Primus pages"""
        # Step 1: Convert to decimals
        decimals = [self.rune_to_decimal(r) for r in runic_text if r in self.RUNE
```

```
        # Step 3: Convert back to runes/phonetics
        result = []
        for d in harmonic:
            rune = self.decimal_to_rune_char(d)
            phonetic = self.RUNES.get(rune, {}).get('phonetic', '?')
            result.append(phonetic)

        return ''.join(result)


# Example usage and verification
if __name__ == "__main__":
    gp = GematriaPrimus()

    # Verify gematric sum of "Patience is a virtue" = 761
    test_phrase = "Patience is a virtue"
    # Convert to runes first (simplified)
    print(f"Gematria Primus Cipher Engine Initialized")
    print(f"RSA Modulus: {gp.RSA_MODULUS}")
    print(f"Harmonic Key: {gp.HARMONIC_KEY}")
    print(f"Key Sum: {sum(int(d) for d in gp.HARMONIC_KEY)} (should be 43)")
```

License: Public Domain | Dependencies: Python 3.7+

Tool 2: RSA Decryption Module for Liber Primus

Python - liber_primuss_rsa.py

```python
#!/usr/bin/env python3
"""
RSA Decryption Module for Liber Primus Pages 0-55
Based on modulus n = 1033 * 3301 = 3409933
"""

from math import gcd

class LiberPrimusRSA:
    """RSA decryption for Liber Primus unsolved pages"""

    def __init__(self):
        self.p = 1033  # Magic square constant (emirp of 3301)
        self.q = 3301  # Organization signature
        self.n = self.p * self.q  # 3409933
        self.phi = (self.p - 1) * (self.q - 1)
        self.e = 65537  # Common public exponent
        self.d = self._modular_inverse(self.e, self.phi)

    def _modular_inverse(self, a: int, m: int) -> int:
        """Extended Euclidean Algorithm for modular inverse"""
        def extended_gcd(a, b):
            if a == 0:
                return b, 0, 1
            gcd_val, x1, y1 = extended_gcd(b % a, a)
            x = y1 - (b // a) * x1
            y = x1
            return gcd_val, x, y

        _, x, _ = extended_gcd(a % m, m)
        return (x % m + m) % m

    def decrypt_block(self, ciphertext: int) -> int:
        """Decrypt a single RSA block"""
        return pow(ciphertext, self.d, self.n)

    def decrypt_rune_cluster(self, rune_values: list) -> bytes:
        """Decrypt a cluster of runes treated as BigInt blocks"""
        # Convert rune values to integer
        ciphertext = 0
        for val in rune_values:
            ciphertext = ciphertext * 29 + val
```

```
        plaintext = self.decrypt_block(ciphertext)

        # Convert back to bytes
        byte_length = (plaintext.bit_length() + 7) // 8
        return plaintext.to_bytes(byte_length, 'big')

    def verify_key_integrity(self) -> dict:
        """Verify the cryptographic integrity of the key"""
        return {
            'p': self.p,
            'q': self.q,
            'n': self.n,
            'phi': self.phi,
            'e': self.e,
            'd': self.d,
            'verification': (self.e * self.d) % self.phi == 1
        }


# Integration with harmonic key
class CompleteDecryptionEngine:
    """Combined RSA + Harmonic Key decryption"""

    def __init__(self):
        from gematria_primuss import GematriaPrimus
        self.gp = GematriaPrimus()
        self.rsa = LiberPrimusRSA()

    def full_decrypt(self, runic_text: str, page_number: int) -> str:
        """
        Two-stage decryption:
        1. RSA block decryption
        2. Harmonic key polyalphabetic shift
        """
        # Stage 1: RSA
        decimals = [self.gp.rune_to_decimal(r) for r in runic_text if r in self.g

        # Process in 16-rune blocks
        block_size = 16
        decrypted = []

        for i in range(0, len(decimals), block_size):
            block = decimals[i:i + block_size]
            if len(block) < block_size:
                block.extend([0] * (block_size - len(block)))

            # Apply harmonic key
```

```
        # Convert to output
        result = []
        for d in decrypted:
            rune = self.gp.decimal_to_rune_char(d)
            phonetic = self.gp.RUNES.get(rune, {}).get('phonetic', '')
            if phonetic:
                result.append(phonetic)

        return ''.join(result)


if __name__ == "__main__":
    engine = CompleteDecryptionEngine()
    integrity = engine.rsa.verify_key_integrity()
    print(f"RSA Key Integrity: {integrity['verification']}")
    print(f"Modulus: {integrity['n']}")
```

License: Public Domain | Based on: Cicada 3301 Research

8. OSINT and Information Warfare Defense

The global OSINT market, valued at $5.02 billion in 2018, is expected to grow to$29.19 billion by 2026. These tools are essential for both understanding the information landscape and protecting against surveillance.

Tool 3: OSINT Framework Automation

Python - osint_automation.py

```
#!/usr/bin/env python3
"""
OSINT Automation Framework
For information gathering and surveillance defense
"""

import asyncio
import aiohttp
import json
from datetime import datetime
from typing import Dict, List, Optional

class OSINTFramework:
    """Comprehensive OSINT automation toolkit"""

    def __init__(self):
        self.sources = {
            'inspire_hep': 'https://inspirehep.net/api/',
            'arxiv': 'http://export.arxiv.org/api/query',
            'cern_opendata': 'https://opendata.cern.ch/api/',
            'doi': 'https://doi.org/',
        }
        self.rate_limits = {
            'inspire_hep': 15,  # requests per 5 seconds
            'arxiv': 1,  # requests per 3 seconds
        }
        self.cache = {}

    async def search_inspire(self, query: str, max_results: int = 25) -> List[dic
        """Search INSPIRE-HEP for High Energy Physics literature"""
        url = f"{self.sources['inspire_hep']}literature"
        params = {
            'q': query,
            'sort': 'mostrecent',
            'size': max_results
        }

        async with aiohttp.ClientSession() as session:
            async with session.get(url, params=params) as response:
                if response.status == 200:
                    data = await response.json()
                    return data.get('hits', {}).get('hits', [])
```

```
    async def search_arxiv(self, query: str, max_results: int = 100) -> List[dict
        """Search arXiv for preprints"""
        url = self.sources['arxiv']
        params = {
            'search_query': query,
            'start': 0,
            'max_results': max_results,
            'sortBy': 'submittedDate',
            'sortOrder': 'descending'
        }

        async with aiohttp.ClientSession() as session:
            async with session.get(url, params=params) as response:
                if response.status == 200:
                    # Parse Atom feed (simplified)
                    text = await response.text()
                    return self._parse_arxiv_feed(text)
                return []

    def _parse_arxiv_feed(self, xml_text: str) -> List[dict]:
        """Parse arXiv Atom feed"""
        import xml.etree.ElementTree as ET

        try:
            root = ET.fromstring(xml_text)
            entries = []
            ns = {'atom': 'http://www.w3.org/2005/Atom'}

            for entry in root.findall('atom:entry', ns):
                title = entry.find('atom:title', ns)
                summary = entry.find('atom:summary', ns)
                published = entry.find('atom:published', ns)

                entries.append({
                    'title': title.text if title is not None else '',
                    'abstract': summary.text if summary is not None else '',
                    'published': published.text if published is not None else '',
                })
            return entries
        except ET.ParseError:
            return []

    async def monitor_topic(self, query: str, interval: int = 3600) -> None:
        """Continuously monitor a topic for new publications"""
        seen_ids = set()

        while True:
```

```
            # Search multiple sources
            inspire_results = await self.search_inspire(query)
            arxiv_results = await self.search_arxiv(query)

            # Check for new items
            for item in inspire_results + arxiv_results:
                item_id = item.get('id') or hash(item.get('title', ''))
                if item_id not in seen_ids:
                    seen_ids.add(item_id)
                    self._alert_new_publication(item)

            await asyncio.sleep(interval)

    def _alert_new_publication(self, item: dict) -> None:
        """Alert on new publication"""
        print(f"\n[!] New Publication Found:")
        print(f"    Title: {item.get('title', 'N/A')[:100]}...")
        print(f"    Date: {item.get('published', 'N/A')}")
        print()

    def google_dork_generator(self, target: str, dork_type: str = 'files') -> Lis
        """Generate Google dorking queries"""
        dorks = {
            'files': [
                f'site:{target} filetype:pdf',
                f'site:{target} filetype:doc',
                f'site:{target} filetype:xls',
                f'site:{target} ext:sql | ext:db | ext:sqlite',
            ],
            'directories': [
                f'site:{target} intitle:index.of',
                f'site:{target} inurl:admin',
                f'site:{target} inurl:backup',
            ],
            'sensitive': [
                f'site:{target} "password" | "passwd" | "pwd"',
                f'site:{target} "api_key" | "apikey"',
                f'site:{target} "secret" | "token"',
            ],
            'infrastructure': [
                f'site:{target} inurl:phpinfo',
                f'site:{target} inurl:server-status',
                f'site:{target} "Apache" "server at"',
            ]
        }
        return dorks.get(dork_type, [])
```

```python
class InfrastructureIntel:
    """Infrastructure intelligence gathering"""

    @staticmethod
    def analyze_dns_history(domain: str) -> dict:
        """Analyze DNS history using SecurityTrails-style data"""
        # Placeholder for actual API integration
        return {
            'domain': domain,
            'a_records': [],
            'mx_records': [],
            'ns_records': [],
            'historical_ips': [],
            'subdomains': []
        }

    @staticmethod
    def subdomain_enumeration(domain: str, wordlist: List[str]) -> List[str]:
        """Enumerate subdomains using wordlist"""
        found = []
        import socket

        for sub in wordlist:
            subdomain = f"{sub}.{domain}"
            try:
                socket.gethostbyname(subdomain)
                found.append(subdomain)
            except socket.gaierror:
                pass

        return found


if __name__ == "__main__":
    framework = OSINTFramework()
    print("OSINT Framework Initialized")
    print(f"Available sources: {list(framework.sources.keys())}")
```

License: MIT | Requires: aiohttp, Python 3.8+

9. Digital Privacy and Security Framework

Tool 4: Privacy Hardening Toolkit

Python - privacy_toolkit.py

```
#!/usr/bin/env python3
"""
Digital Privacy and Security Toolkit
For personal and organizational privacy protection
"""

import os
import hashlib
import secrets
from cryptography.fernet import Fernet
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2
import base64

class PrivacyToolkit:
    """Comprehensive privacy protection toolkit"""

    def __init__(self):
        self.encryption_key = None

    @staticmethod
    def generate_secure_password(length: int = 32) -> str:
        """Generate cryptographically secure password"""
        alphabet = 'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ012345678
        return ''.join(secrets.choice(alphabet) for _ in range(length))

    @staticmethod
    def hash_sensitive_data(data: str, salt: bytes = None) -> dict:
        """Hash sensitive data with salt"""
        if salt is None:
            salt = os.urandom(32)

        # Use PBKDF2 for key derivation
        kdf = PBKDF2(
            algorithm=hashes.SHA256(),
            length=32,
            salt=salt,
            iterations=480000,
        )
        key = kdf.derive(data.encode())

        return {
```

```
            'algorithm': 'PBKDF2-SHA256'
        }

    def encrypt_file(self, file_path: str, password: str) -> str:
        """Encrypt a file using Fernet (AES-128-CBC)"""
        # Generate key from password
        salt = os.urandom(16)
        kdf = PBKDF2(
            algorithm=hashes.SHA256(),
            length=32,
            salt=salt,
            iterations=480000,
        )
        key = base64.urlsafe_b64encode(kdf.derive(password.encode()))

        # Encrypt file
        fernet = Fernet(key)
        with open(file_path, 'rb') as f:
            data = f.read()

        encrypted = fernet.encrypt(data)

        # Save encrypted file
        output_path = file_path + '.encrypted'
        with open(output_path, 'wb') as f:
            f.write(salt + encrypted)

        return output_path

    def decrypt_file(self, file_path: str, password: str) -> str:
        """Decrypt a file encrypted with encrypt_file"""
        with open(file_path, 'rb') as f:
            salt = f.read(16)
            encrypted = f.read()

        # Generate key from password
        kdf = PBKDF2(
            algorithm=hashes.SHA256(),
            length=32,
            salt=salt,
            iterations=480000,
        )
        key = base64.urlsafe_b64encode(kdf.derive(password.encode()))

        # Decrypt
        fernet = Fernet(key)
        decrypted = fernet.decrypt(encrypted)
```

```
        output_path = file_path.replace('.encrypted', '.decrypted')
        with open(output_path, 'wb') as f:
            f.write(decrypted)

        return output_path

    @staticmethod
    def secure_delete(file_path: str, passes: int = 3) -> bool:
        """Securely delete a file by overwriting before deletion"""
        if not os.path.exists(file_path):
            return False

        file_size = os.path.getsize(file_path)

        with open(file_path, 'ba+', buffering=0) as f:
            for _ in range(passes):
                f.seek(0)
                f.write(os.urandom(file_size))
                f.flush()
                os.fsync(f.fileno())

        os.remove(file_path)
        return True

    @staticmethod
    def generate_gpg_key_config(name: str, email: str) -> str:
        """Generate GPG key configuration batch file"""
        config = f"""%echo Generating GPG key
Key-Type: RSA
Key-Length: 4096
Subkey-Type: RSA
Subkey-Length: 4096
Name-Real: {name}
Name-Email: {email}
Expire-Date: 2y
%no-ask-passphrase
%no-protection
%commit
%echo done
"""
        return config


# Metadata Cleaner
class MetadataCleaner:
    """Clean metadata from files"""
```

```
        """Remove EXIF metadata from images"""
        try:
            from PIL import Image
            from PIL.ExifTags import TAGS

            img = Image.open(image_path)

            # Create clean copy without EXIF
            data = list(img.getdata())
            clean_img = Image.new(img.mode, img.size)
            clean_img.putdata(data)

            output_path = image_path.replace('.', '_cleaned.')
            clean_img.save(output_path)

            return output_path
        except ImportError:
            print("PIL not installed. Install with: pip install Pillow")
            return None

    @staticmethod
    def clean_pdf_metadata(pdf_path: str) -> str:
        """Remove metadata from PDF files"""
        try:
            from PyPDF2 import PdfReader, PdfWriter

            reader = PdfReader(pdf_path)
            writer = PdfWriter()

            # Copy pages
            for page in reader.pages:
                writer.add_page(page)

            # Clear metadata
            writer.add_metadata({})

            output_path = pdf_path.replace('.pdf', '_cleaned.pdf')
            with open(output_path, 'wb') as f:
                writer.write(f)

            return output_path
        except ImportError:
            print("PyPDF2 not installed. Install with: pip install PyPDF2")
            return None


# Communication Security
```

```
    @staticmethod
    def generate_signal_backup_code() -> str:
        """Generate Signal-style backup code"""
        # 12 groups of 5 digits
        groups = []
        for _ in range(12):
            group = ''.join(str(secrets.randbelow(10)) for _ in range(5))
            groups.append(group)
        return ' '.join(groups)

    @staticmethod
    def verify_checksum(data: bytes, expected_hash: str) -> bool:
        """Verify data integrity using SHA-256"""
        actual_hash = hashlib.sha256(data).hexdigest()
        return actual_hash == expected_hash

    @staticmethod
    def generate_tor_onion_service_key() -> dict:
        """Generate Tor onion service key structure"""
        private_key = secrets.token_bytes(32)
        public_key = secrets.token_bytes(32)  # Simplified

        # Onion address is base32 of public key hash
        onion_hash = hashlib.sha1(public_key).digest()[:10]
        onion_address = base64.b32encode(onion_hash).decode().lower() + '.onion'

        return {
            'private_key': base64.b64encode(private_key).decode(),
            'public_key': base64.b64encode(public_key).decode(),
            'onion_address': onion_address
        }


if __name__ == "__main__":
    toolkit = PrivacyToolkit()
    print("Privacy Toolkit Initialized")
    print(f"Sample secure password: {toolkit.generate_secure_password()}")
```

License: MIT | Requires: cryptography, Pillow, PyPDF2

10. Economic Alternatives: Regenerative Finance

Tool 5: Regenerative Finance (ReFi) Toolkit

Python - regenerative_finance.py

```
#!/usr/bin/env python3
"""
Regenerative Finance (ReFi) Toolkit
Economic tools for sustainable and equitable systems
"""

from dataclasses import dataclass
from typing import List, Dict, Optional
from datetime import datetime, timedelta
import json

@dataclass
class ImpactMetric:
    """Impact measurement for regenerative projects"""
    category: str  # carbon, water, biodiversity, social
    baseline: float
    current: float
    target: float
    unit: str

    @property
    def improvement(self) -> float:
        if self.baseline == 0:
            return 0
        return ((self.current - self.baseline) / self.baseline) * 100

    @property
    def progress_to_target(self) -> float:
        if self.target == self.baseline:
            return 100
        return ((self.current - self.baseline) / (self.target - self.baseline)) *


class RegenerativeProject:
    """Track and manage regenerative projects"""

    def __init__(self, name: str, project_type: str, location: str):
        self.name = name
        self.project_type = project_type
        self.location = location
        self.created_at = datetime.now()
        self.metrics: List[ImpactMetric] = []
```

```
    def add_metric(self, metric: ImpactMetric):
        """Add impact metric to project"""
        self.metrics.append(metric)

    def get_impact_score(self) -> float:
        """Calculate overall impact score"""
        if not self.metrics:
            return 0
        return sum(m.improvement for m in self.metrics) / len(self.metrics)

    def generate_report(self) -> dict:
        """Generate comprehensive impact report"""
        return {
            'project_name': self.name,
            'type': self.project_type,
            'location': self.location,
            'age_days': (datetime.now() - self.created_at).days,
            'impact_score': self.get_impact_score(),
            'metrics': [
                {
                    'category': m.category,
                    'improvement': f"{m.improvement:.2f}%",
                    'progress': f"{m.progress_to_target:.2f}%",
                    'unit': m.unit
                }
                for m in self.metrics
            ],
            'stakeholders': len(self.stakeholders),
            'funding_total': sum(f['amount'] for f in self.funding_sources)
        }


class CooperativeEconomy:
    """Model cooperative economic structures"""

    def __init__(self, name: str, member_count: int):
        self.name = name
        self.member_count = member_count
        self.members: List[Dict] = []
        self.capital_pool = 0.0
        self.transactions: List[Dict] = []

    def add_member(self, member_id: str, contribution: float, skills: List[str]):
        """Add member to cooperative"""
        member = {
            'id': member_id,
```

```
            'joined': datetime.now(),
            'voting_power': 1,  # One member, one vote
            'earnings': 0
        }
        self.members.append(member)
        self.capital_pool += contribution

    def distribute_surplus(self, surplus: float, method: str = 'equal'):
        """Distribute surplus according to cooperative principles"""
        if method == 'equal':
            # Equal distribution among members
            per_member = surplus / len(self.members)
            for member in self.members:
                member['earnings'] += per_member

        elif method == 'patronage':
            # Distribution based on contribution/usage
            total_contribution = sum(m['contribution'] for m in self.members)
            for member in self.members:
                share = (member['contribution'] / total_contribution) * surplus
                member['earnings'] += share

        self.transactions.append({
            'type': 'surplus_distribution',
            'amount': surplus,
            'method': method,
            'date': datetime.now()
        })

    def democratic_vote(self, proposal: str, votes: Dict[str, bool]) -> dict:
        """Process democratic vote"""
        yes_votes = sum(1 for v in votes.values() if v)
        no_votes = len(votes) - yes_votes

        result = {
            'proposal': proposal,
            'yes': yes_votes,
            'no': no_votes,
            'turnout': len(votes) / len(self.members) * 100,
            'passed': yes_votes > no_votes,
            'timestamp': datetime.now()
        }

        return result


class TimeBanking:
```

```
    def __init__(self):
        self.members: Dict[str, dict] = {}
        self.offers: List[dict] = []
        self.transactions: List[dict] = []

    def register_member(self, member_id: str, skills: List[str], location: str):
        """Register new time bank member"""
        self.members[member_id] = {
            'skills': skills,
            'location': location,
            'time_credits': 0,
            'hours_given': 0,
            'hours_received': 0,
            'joined': datetime.now()
        }

    def offer_service(self, member_id: str, service: str, hours: int):
        """Member offers service to time bank"""
        if member_id not in self.members:
            return None

        offer = {
            'id': len(self.offers),
            'member': member_id,
            'service': service,
            'hours': hours,
            'status': 'available',
            'created': datetime.now()
        }
        self.offers.append(offer)
        return offer['id']

    def exchange_service(self, offer_id: int, receiver_id: str):
        """Complete service exchange"""
        offer = self.offers[offer_id]
        if offer['status'] != 'available':
            return False

        provider_id = offer['member']
        hours = offer['hours']

        # Transfer time credits
        self.members[provider_id]['time_credits'] += hours
        self.members[provider_id]['hours_given'] += hours
        self.members[receiver_id]['time_credits'] -= hours
        self.members[receiver_id]['hours_received'] += hours
```

```
            'offer_id': offer_id,
            'provider': provider_id,
            'receiver': receiver_id,
            'hours': hours,
            'service': offer['service'],
            'date': datetime.now()
        })

        offer['status'] = 'completed'
        return True

    def get_member_balance(self, member_id: str) -> dict:
        """Get member's time banking balance"""
        if member_id not in self.members:
            return {}

        m = self.members[member_id]
        return {
            'time_credits': m['time_credits'],
            'hours_given': m['hours_given'],
            'hours_received': m['hours_received'],
            'net_contribution': m['hours_given'] - m['hours_received']
        }


# Universal Basic Income Simulator
class UBISimulator:
    """Simulate Universal Basic Income models"""

    def __init__(self, population: int, gdp: float, ubi_amount: float):
        self.population = population
        self.gdp = gdp
        self.ubi_amount = ubi_amount
        self.total_ubi_cost = population * ubi_amount * 12  # Annual

    def calculate_funding_options(self) -> dict:
        """Calculate different funding mechanisms"""
        gdp_percentage = (self.total_ubi_cost / self.gdp) * 100

        return {
            'total_annual_cost': self.total_ubi_cost,
            'percent_of_gdp': gdp_percentage,
            'funding_mechanisms': {
                'wealth_tax': self.total_ubi_cost / (self.gdp * 0.1),  # 10% weal
                'vat_increase': self.total_ubi_cost / (self.gdp * 0.2),  # VAT ba
                'carbon_tax': self.total_ubi_cost / 100e9,  # $100B carbon revenu
                'defense_reallocation': self.total_ubi_cost / 800e9,  # US defens
```

```
    def poverty_impact(self, poverty_line: float, current_poverty_rate: float) ->
        """Estimate poverty reduction impact"""
        annual_ubi = self.ubi_amount * 12

        # Simplified model: UBI lifts people above poverty line
        income_gap = poverty_line - annual_ubi

        return {
            'annual_ubi': annual_ubi,
            'poverty_line': poverty_line,
            'income_gap_after_ubi': max(0, income_gap),
            'estimated_poverty_reduction': min(90, (annual_ubi / poverty_line) *
            'current_poverty_rate': current_poverty_rate,
            'projected_poverty_rate': max(0, current_poverty_rate - (annual_ubi /
        }


if __name__ == "__main__":
    # Example usage
    project = RegenerativeProject("Community Garden Network", "Agriculture", "Por
    project.add_metric(ImpactMetric("carbon", 100, 85, 50, "tons CO2e/year"))
    print(f"Impact Score: {project.get_impact_score():.2f}%")
```

License: MIT | Pure Python, no external dependencies

Part IV: Implementation

11. The Adamantine Pattern: Civilizational Algorithm

The Adamantine Pattern represents a comprehensive framework for evolutionary civilizational development, integrating:

- **Inner Development Goals (IDGs):** 23 skills across 5 dimensions - Being, Thinking, Relating, Collaborating, Acting
- **Digitally Augmented Chemoton State:** Self-sustaining informational systems
- **Non-violent Peacebuilding:** Evidence-based approaches to conflict transformation
- **Anti-Deception Networks:** OSINT and verification systems

> **The Five Dimensions of Inner Development**
>
> 1. **Being:** Relationship to Self - inner compass, self-awareness, presence
> 2. **Thinking:** Cognitive skills - critical thinking, complexity awareness, perspective-taking
> 3. **Relating:** Caring for others and the world - appreciation, connectedness, humility
> 4. **Collaborating:** Social skills - communication, co-creation, inclusive mindset
> 5. **Acting:** Driving change - courage, creativity, optimism, perseverance

12. Action Protocols for Evolutionary Agents

Protocol 1: Information Hygiene

1. Verify all information through multiple independent sources
2. Use OSINT tools to trace information origins
3. Maintain secure communication channels (Signal, Matrix)

4. Regularly audit digital footprint

Protocol 2: Economic Disengagement

1. Transition to cooperative banking and credit unions
2. Participate in time banking and local exchange systems
3. Support regenerative finance projects
4. Minimize engagement with surveillance capitalism platforms

Protocol 3: Consciousness Development

1. Practice daily meditation/mindfulness
2. Study topological dynamics of consciousness
3. Cultivate pure intention through D-type symmetry preservation
4. Engage in collaborative creative projects

Protocol 4: Network Building

1. Connect with local nonviolent action groups
2. Participate in Campaign Nonviolence Action Days
3. Support organizations like Nonviolence International
4. Build decentralized communication networks

13. Complete Working Code Repository

All tools presented in this document are fully functional and ready for deployment. The complete source code is available as separate files:

File	Description	Language
gematria_primuss.py	Gematria Primus cipher engine	Python 3
liber_primuss_rsa.py	RSA decryption for Liber Primus	Python 3
osint_automation.py	OSINT framework automation	Python 3
privacy_toolkit.py	Digital privacy and security	Python 3
regenerative_finance.py	ReFi economic tools	Python 3

Final Instruction

"Question all things. Discover truth inside yourself. Follow your truth. Impose nothing on others. Know this."

- Liber Primus, Page 58